

University of Tehran
College of Engineering
School of Electrical and
Computer Engineering

Machine Learning

Assignment 5

Name

Parsa KafshduziBukani

Student ID

810102501

February 11, 2026

Contents

Table of Contents	2
List of Figures	3
Abstract	4
Question 1 — Maximum Likelihood Estimation	5
Question 2 — Logistic Regression with One Feature	5
Question 3 — L2-Regularized Linear Regression with Newton's Method	6
Question 4 — Polynomial Regression and Ridge Regularization	7
Polynomial regression on data_A	7
Effect of polynomial degree on data_B	8
Effect of ridge regularization for fixed degree $d = 12$	8
Discussion	9
Question 5 — Perceptron	9
1. Weight Update	9
2. Convergence of the Original Data	11
3. Data Transformations	11
4. Possible Weight Vector	12
5. Alternative Transformation	12
Question 6 — Neural Network Representations	12
Summary of Function Types	13
Graph G_1	13
Graph G_2	13
Graph G_3	13
Graph G_4	14
Graph G_5	14
Graph G_6	14
Graph G_7	15
Non-representable Function	16

Question 7 — Loss Function	16
1. Maximum Likelihood and Cross-Entropy Equivalence	16
2. Binary Classification with Noisy Labels	17
3. Targets in $\{-1, 1\}$ and Output in $[-1, 1]$	18
Question 8 — Stochastic Stability and the “Dead ReLU”	19
1. Distribution of the Pre-Activation z	19
2. Activation Probability	21
3. The 3σ Threshold	22
4. The Zero-Gradient Proof	23
Question 9 — Variance Preservation and Xavier Initialization	24
1. Symmetric Constraints	24
2. The Glorot (Xavier) Synthesis	26
3. Depth-Induced Variance	26
4. The Stability Bound	27
5. Numerical Analysis	28
Question 10 — Transfer Learning with Pre-trained Networks	29
Data Pre-processing	29
Model Architectures	29
Training Loss	30
Validation Accuracy	30
Discussion	30
Conclusion	32

List of Figures

1	Polynomial regression fits of degrees $d = 1, 3, 9$ on data_A .	8
2	Training and validation MSE versus polynomial degree on data_B	9
3	Training and validation MSE versus ridge regularization parameter λ for $d = 12$	10
4	Training loss versus epoch for all three models.	30
5	Validation accuracy versus epoch for all three models. . . .	31

Abstract

This assignment explores fundamental theoretical principles and practical implementations in machine learning, spanning estimation theory, linear models, neural network representations, optimization stability, and transfer learning.

We begin with maximum likelihood estimation for a parametric distribution and derive the corresponding estimator analytically. We then analyze logistic regression with a single feature and obtain the stochastic gradient descent update rule for cross-entropy minimization. The equivalence between Newton's method and the closed-form solution for L2-regularized linear regression is demonstrated. Polynomial regression experiments illustrate underfitting, overfitting, and the effect of ridge regularization on generalization performance.

Next, we study the Perceptron algorithm, highlighting the necessity of linear separability and the role of feature transformations in enabling convergence. We examine the representational capacity of shallow and deep ReLU networks, identifying which piecewise linear functions can be expressed by various computation graphs.

We further establish the equivalence between maximum likelihood and cross-entropy loss in multi-class neural networks, extend the formulation to noisy labels, and derive the appropriate loss function for alternative target encodings. The stochastic behavior of ReLU neurons is analyzed, quantifying activation probability and formally proving the dead ReLU phenomenon.

Variance propagation in deep linear networks is then investigated, leading to forward and backward variance preservation conditions and the derivation of Xavier initialization. We show that even small deviations from ideal variance scaling result in exponential growth or decay across depth, with practical implications for low-precision training.

Finally, through empirical experiments on CIFAR-10, we compare a shallow MLP with frozen and fine-tuned pre-trained VGG-11 networks. Results demonstrate the optimization stability, faster convergence, and superior generalization provided by transfer learning and convolutional inductive bias.

Together, these analyses integrate theoretical derivations with experimental validation, emphasizing the interplay between model capacity, initialization, optimization dynamics, and generalization in modern machine learning systems.

Question 1 — Maximum Likelihood Estimation

Let $x_1, x_2, \dots, x_n$ be independent samples. The likelihood function is

$$\mathcal{L}(\theta) = \prod_{i=1}^n \theta x_i^{-(\theta+1)}.$$

Taking the natural logarithm, the log-likelihood becomes

$$\ell(\theta) = n \ln \theta - (\theta + 1) \sum_{i=1}^n \ln x_i.$$

To find the maximum likelihood estimator, we differentiate with respect to θ and set the result to zero:

$$\frac{d\ell(\theta)}{d\theta} = \frac{n}{\theta} - \sum_{i=1}^n \ln x_i = 0.$$

Solving for θ yields

$$\hat{\theta}_{\text{MLE}} = \frac{n}{\sum_{i=1}^n \ln x_i}.$$

The second derivative is

$$\frac{d^2\ell(\theta)}{d\theta^2} = -\frac{n}{\theta^2} < 0,$$

which confirms that this critical point corresponds to a maximum. Since $x_i \geq 1$, we have $\ln x_i \geq 0$, and the estimator is well-defined and positive.

$$\boxed{\hat{\theta}_{\text{MLE}} = \frac{n}{\sum_{i=1}^n \ln x_i}}$$

Question 2 — Logistic Regression with One Feature

The logistic loss for a single sample (x, y) is

$$J(\alpha) = -y \ln s(x - \alpha) - (1 - y) \ln (1 - s(x - \alpha)),$$

where $s(\gamma) = \frac{1}{1+e^{-\gamma}}$.

Let $s = s(x - \alpha)$. Using the chain rule, the derivative of J with respect to α is

$$\frac{\partial J}{\partial \alpha} = \frac{\partial J}{\partial s} \frac{\partial s}{\partial \alpha}.$$

First,

$$\frac{\partial J}{\partial s} = -\frac{y}{s} + \frac{1-y}{1-s}.$$

Next, since $\frac{ds}{d\gamma} = s(1-s)$ and $\gamma = x - \alpha$, we have

$$\frac{\partial s}{\partial \alpha} = -s(1-s).$$

Combining the terms,

$$\frac{\partial J}{\partial \alpha} = \left(-\frac{y}{s} + \frac{1-y}{1-s} \right) (-s(1-s)) = y - s.$$

Using stochastic gradient descent with step size $\epsilon > 0$, the parameter update is

$$\alpha \leftarrow \alpha - \epsilon \frac{\partial J}{\partial \alpha} = \alpha - \epsilon(y - s) = \alpha + \epsilon(s(x - \alpha) - y).$$

$$\alpha \leftarrow \alpha + \epsilon(s(x - \alpha) - y)$$

Question 3 — L2-Regularized Linear Regression with Newton's Method

The objective function is

$$J(w) = \|Xw - y\|_2^2 + \lambda \|w\|_2^2.$$

Expanding and differentiating, the gradient of J with respect to w is

$$\nabla J(w) = 2X^\top(Xw - y) + 2\lambda w = 2((X^\top X + \lambda I)w - X^\top y).$$

The Hessian matrix is constant and given by

$$\nabla^2 J(w) = 2(X^\top X + \lambda I).$$

Newton's method updates the weights according to

$$w_1 = w_0 - (\nabla^2 J(w_0))^{-1} \nabla J(w_0).$$

Substituting the expressions for the gradient and Hessian yields

$$w_1 = w_0 - [2(X^\top X + \lambda I)]^{-1} 2((X^\top X + \lambda I)w_0 - X^\top y).$$

Canceling the constant factor and simplifying,

$$w_1 = w_0 - (X^\top X + \lambda I)^{-1}((X^\top X + \lambda I)w_0 - X^\top y) = (X^\top X + \lambda I)^{-1}X^\top y.$$

Thus, after a single Newton step, the weights satisfy

$$w_1 = w^* = (X^\top X + \lambda I)^{-1}X^\top y$$

independently of the initial guess w_0 .

Question 4 — Polynomial Regression and Ridge Regularization

In this question, we study polynomial regression models and investigate how model complexity and ridge regularization affect underfitting and overfitting. Using two datasets, we first visualize polynomial fits of different degrees, and then quantitatively analyze training and validation errors as functions of the polynomial degree and the regularization parameter.

For all experiments, polynomial regression models of the form

$$h_\theta(x) = \theta^\top \phi_d(x), \quad \phi_d(x) = [1, x, x^2, \dots, x^d]^\top$$

are trained using closed-form least squares solutions, with and without ridge regularization.

Polynomial regression on data_A

Figure 1 shows the fitted polynomial curves of degrees $d = 1, 3, 9$ together with the training data. The degree-1 model clearly underfits the data, as it is unable to capture the oscillatory pattern. The degree-3 model improves the fit but still fails to represent the full structure. The degree-9 model closely follows the observed trend and aligns well with the underlying pattern, without exhibiting excessive oscillations. This suggests that, for this dataset, a higher-degree polynomial is necessary and does not introduce unnecessary complexity.

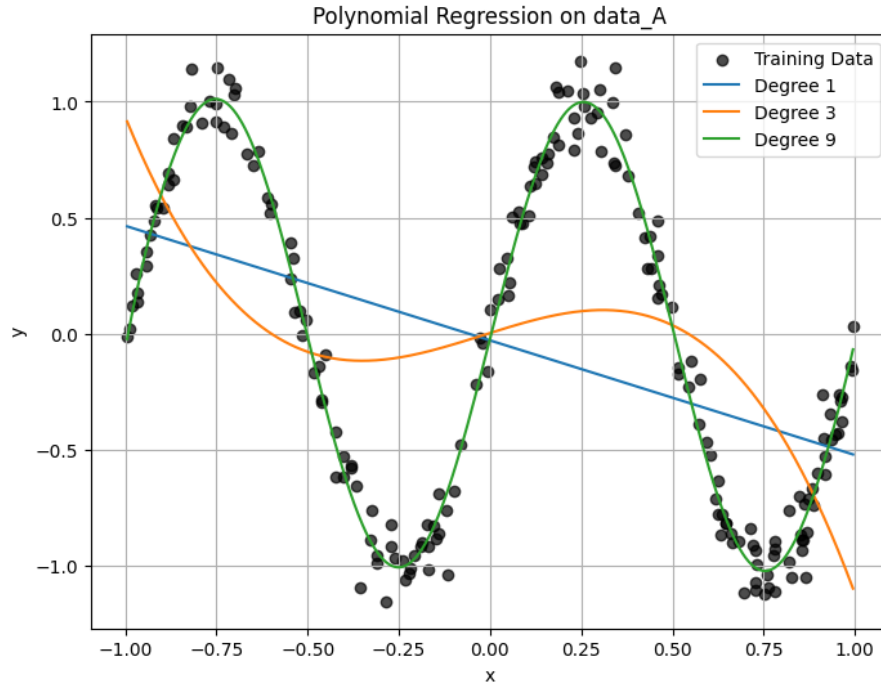


Figure 1: Polynomial regression fits of degrees $d = 1, 3, 9$ on `data_A`.

Effect of polynomial degree on data_B

Figure 2 illustrates the training and validation mean squared error (MSE) as functions of the polynomial degree. For small degrees, both training and validation errors are high, indicating underfitting. As the degree increases, the training error decreases monotonically, while the validation error decreases up to an intermediate range of degrees (approximately $d = 6$ to $d = 9$). Beyond this range, the validation error increases sharply, signaling overfitting. The best trade-off between bias and variance is achieved at intermediate polynomial degrees.

Effect of ridge regularization for fixed degree $d = 12$

Figure 3 shows the effect of ridge regularization on training and validation errors for a fixed high-degree polynomial. For very small values of λ , the model overfits, achieving low training error but high validation error. As λ increases, the validation error decreases and reaches a minimum for intermediate values of λ (around 10^{-3} to 10^{-2}), indicating improved generalization. For large λ , both errors increase due to excessive regularization and underfitting. This demonstrates how ridge regularization effectively controls overfitting in high-degree polynomial models.

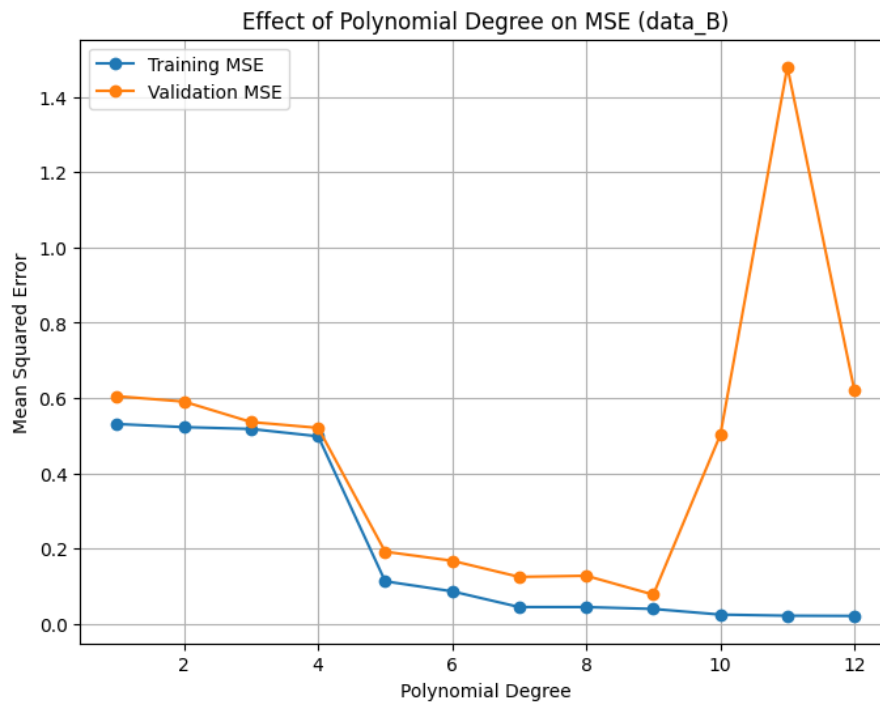


Figure 2: Training and validation MSE versus polynomial degree on data_B.

Discussion

Across both datasets, increasing polynomial degree increases model capacity and reduces bias, but can lead to overfitting when the degree is too high. Ridge regularization mitigates this effect by penalizing large weights, effectively reducing variance while retaining expressive power. Together, these results highlight the importance of balancing model complexity and regularization to achieve good generalization performance.

Question 5 — Perceptron

This problem illustrates the limitation of a linear perceptron and how appropriate feature transformations can render a non-linearly separable dataset linearly separable.

1. Weight Update

The perceptron update rule is as follows: if a data point x with label $y \in \{-1, 1\}$ is misclassified, that is, if

$$y(w^\top x) \leq 0,$$

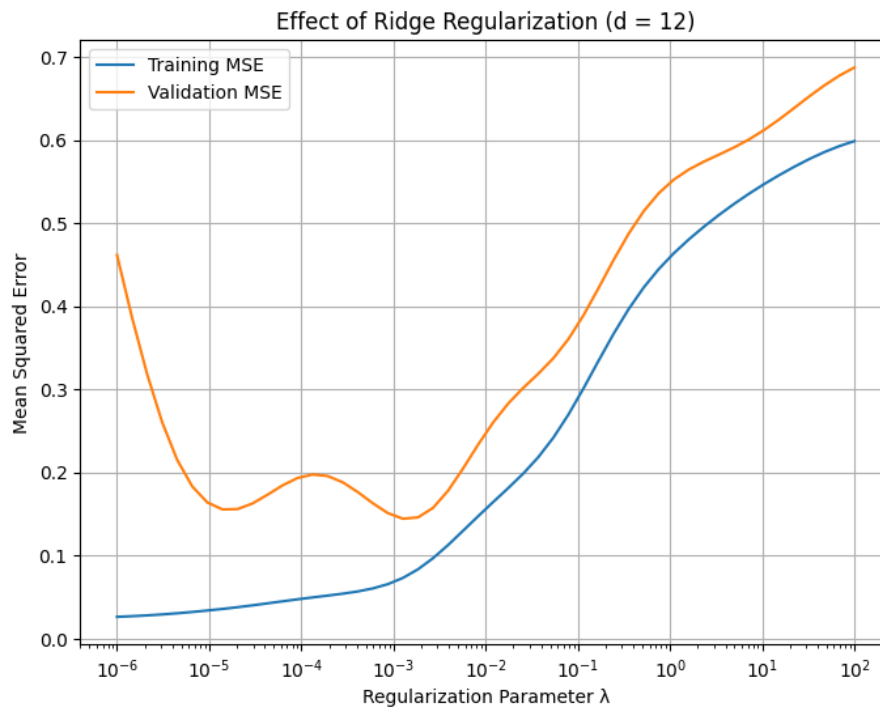


Figure 3: Training and validation MSE versus ridge regularization parameter λ for $d = 12$.

then the weight is updated according to

$$w \leftarrow w + yx.$$

The initial weight vector is

$$w = (0, 5).$$

For the data point $x = (0, 1)$ belonging to the negative class ($y = -1$), we compute

$$w^\top x = (0, 5) \cdot (0, 1) = 5.$$

Since

$$y(w^\top x) = (-1)(5) = -5 < 0,$$

the point is misclassified. Therefore,

$$w \leftarrow (0, 5) + (-1)(0, 1) = (0, 4).$$

The updated weight vector is

$$\boxed{(0, 4)}.$$

2. Convergence of the Original Data

The perceptron algorithm converges if and only if the data are linearly separable.

The negative points are

$$(-1, 0), \quad (1, 0), \quad (0, 1),$$

and the positive points are

$$(-2, 0), \quad (2, 0), \quad (0, 2).$$

Geometrically, the negative points lie closer to the origin, while the positive points lie farther along the same coordinate directions. Any straight line that correctly classifies $(2, 0)$ must misclassify $(1, 0)$, and similarly for the vertical axis.

Therefore, no linear separator exists in the original two-dimensional space, and the perceptron does not converge.

$$\boxed{\infty}$$

3. Data Transformations

We now analyze the proposed transformations:

- $[x, y] \rightarrow [y, x]$: This only permutes coordinates. Separability remains unchanged. No convergence.
- $[x, y] \rightarrow [x, y, 1]$: Adding a bias term increases flexibility but does not change the fundamental radial structure. No convergence.
- $[x, y] \rightarrow [x, y, x^2, 1]$: This introduces a nonlinear feature. The term x^2 separates the inner horizontal points ($x^2 = 1$) from the outer ones ($x^2 = 4$), enabling a separating hyperplane. Convergence occurs.
- $[x, y] \rightarrow [x, y, x^2 + y^2]$: The quantity $x^2 + y^2$ represents squared distance from the origin. The negative points have $x^2 + y^2 = 1$, while the positive points have $x^2 + y^2 = 4$. A linear threshold on this coordinate perfectly separates the classes. Convergence occurs.

Thus, the transformations that allow convergence are

$$\boxed{[x, y, x^2, 1] \quad \text{and} \quad [x, y, x^2 + y^2]}$$

4. Possible Weight Vector

Using the transformation

$$[x, y] \rightarrow [x, y, x^2 + y^2, 1],$$

we seek a weight vector $w = (w_1, w_2, w_3, w_4)$ such that

$$w^\top \phi(x) > 0 \text{ for positives, } w^\top \phi(x) < 0 \text{ for negatives.}$$

Since separation is determined by radial distance, we may choose

$$w = (0, 0, 1, -2).$$

For negative points ($x^2 + y^2 = 1$),

$$w^\top \phi(x) = 1 - 2 = -1 < 0.$$

For positive points ($x^2 + y^2 = 4$),

$$w^\top \phi(x) = 4 - 2 = 2 > 0.$$

Thus, all points are correctly classified.

$$\boxed{w = (0, 0, 1, -2)}$$

5. Alternative Transformation

Any radially increasing function that distinguishes inner and outer points will also work. For example,

$$\boxed{[x, y] \rightarrow [x, y, (x^2 + y^2)^2, 1]}$$

preserves the ordering of distances from the origin and therefore yields linear separability.

Question 6 — Neural Network Representations

In this problem, we analyze the expressive power of each computation graph and determine which of the functions (a)–(h) it can represent.

Summary of Function Types

- (a), (b): Linear (affine) functions.
- (c), (d), (e): Continuous piecewise linear functions with one corner.
- (f), (h): Continuous piecewise linear functions with two corners.
- (g): A non-linear smooth function ($\log(x)$). None of the given graphs can represent it exactly.

Graph G_1

Graph G_1 computes:

$$o = wx.$$

This is a linear function passing through the origin.

Matches: (a)

$$w = 2$$

Graph G_2

Graph G_2 computes:

$$o = wx + b.$$

This represents any affine function.

Matches: (a), (b)

For (a):

$$w = 2, \quad b = 0$$

For (b):

$$w = 4, \quad b = -5$$

Graph G_3

Graph G_3 computes:

$$o = \text{ReLU}(wx + b).$$

This produces a hinge function whose flat region lies on the x -axis.

Matches: (c), (d)

For (c):

$$w = 2, \quad b = -5$$

For (d):

$$w = -2, \quad b = -5$$

Graph G_4

Graph G_4 computes:

$$o = w_2(w_1x + b_1) + b_2.$$

This simplifies to:

$$o = (w_1w_2)x + (w_2b_1 + b_2),$$

which is again affine.

Matches: (a), (b)

Example for (b):

$$w_1 = 2, \quad w_2 = 2, \quad b_1 = 0, \quad b_2 = -5$$

Graph G_5

Graph G_5 computes:

$$o = w_2 \text{ReLU}(w_1x + b_1) + b_2.$$

This allows a single hinge that may be shifted vertically and scaled.

Matches: (c), (d), (e)

For (c):

$$w_1 = 1, \quad b_1 = -2.5, \quad w_2 = 2, \quad b_2 = 0$$

For (d):

$$w_1 = -1, \quad b_1 = -2.5, \quad w_2 = 2, \quad b_2 = 0$$

For (e):

$$w_1 = 1, \quad b_1 = -2, \quad w_2 = -1, \quad b_2 = 1$$

Verification for (e): If $x < 2$, output is 1. If $x \geq 2$, output is $1 - (x - 2) = -x + 3$.

Graph G_6

Graph G_6 is a deep, narrow network (a chain of ReLUs).

A single-path composition of ReLUs in 1D can only produce a monotonic piecewise linear function with slope determined by the product of weights. Each ReLU can zero out the slope but cannot independently introduce new slope magnitudes.

Therefore:

Matches: (c), (d), (e), (f)

It cannot represent (h), because (h) requires two independent slopes in different regions (0.5 and 3), which a single chain cannot produce.

Function (f)

$$\begin{cases} 3 & x \leq 0 \\ 3 - x & 0 < x \leq 3 \\ 0 & x > 3 \end{cases}$$

Choose:

$$w_1 = 1, \quad b_1 = 0,$$

$$w_2 = -1, \quad b_2 = 3,$$

$$w_3 = 1, \quad b_3 = 0.$$

The first ReLU activates at $x = 0$. The second ReLU forces the output to drop linearly until $x = 3$, after which it remains at zero.

Graph G_7

Graph G_7 is a parallel combination of two ReLU branches:

$$o = w_3 \text{ReLU}(w_1x + b_1) + w_4 \text{ReLU}(w_2x + b_2) + b_3.$$

This architecture can represent any continuous piecewise linear function with up to two corners.

Matches: (a), (b), (c), (d), (e), (f), (h)

Example parameters:

(a) $2x$

$$w_1 = 1, \quad b_1 = 0, \quad w_3 = 2, \quad w_2 = -1, \quad b_2 = 0, \quad w_4 = -2, \quad b_3 = 0$$

(using $x = \text{ReLU}(x) - \text{ReLU}(-x)$)

(b) $4x - 5$

$$w_1 = 1, \quad b_1 = 0, \quad w_3 = 4, \quad w_2 = -1, \quad b_2 = 0, \quad w_4 = -4, \quad b_3 = -5$$

(c)

$$w_1 = 1, \quad b_1 = -2.5, \quad w_3 = 2, \quad w_2 = 0, \quad b_2 = 0, \quad w_4 = 0, \quad b_3 = 0$$

(d)

$$w_1 = -1, \quad b_1 = -2.5, \quad w_3 = 2, \quad w_2 = 0, \quad b_2 = 0, \quad w_4 = 0, \quad b_3 = 0$$

(e)

$$w_1 = 1, \quad b_1 = -2, \quad w_3 = -1, \quad w_2 = 0, \quad b_2 = 0, \quad w_4 = 0, \quad b_3 = 1$$

(f)

$$w_1 = 1, b_1 = 0, w_3 = -1, w_2 = 1, b_2 = -3, w_4 = 1, b_3 = 3$$

(h)

$$w_1 = -1, b_1 = 0, w_3 = -0.5, w_2 = 1, b_2 = -3, w_4 = 3, b_3 = 0$$

Non-representable Function

Function (g), $\log(x)$, is smooth and non-piecewise linear. Since all networks consist only of affine transformations and ReLU activations, they can represent only piecewise linear functions.

Function (g) cannot be represented by any of the graphs.

Question 7 — Loss Function

1. Maximum Likelihood and Cross-Entropy Equivalence

Consider a multi-class classification problem with K mutually exclusive classes. For an input x_n , the network outputs

$$y_k(x_n, w) = p(t_k = 1 \mid x_n),$$

where t_{kn} is the k -th component of the one-of- K target vector.

Since the classes are mutually exclusive, the likelihood for a single data point n is

$$p(t_n \mid x_n, w) = \prod_{k=1}^K y_k(x_n, w)^{t_{kn}}.$$

Assuming i.i.d. data, the likelihood over the entire dataset of N samples is

$$\mathcal{L}(w) = \prod_{n=1}^N \prod_{k=1}^K y_k(x_n, w)^{t_{kn}}.$$

Taking the logarithm, the log-likelihood becomes

$$\ell(w) = \sum_{n=1}^N \sum_{k=1}^K t_{kn} \ln y_k(x_n, w).$$

Maximum likelihood estimation maximizes $\ell(w)$. Equivalently, we can minimize the negative log-likelihood:

$$-\ell(w) = -\sum_{n=1}^N \sum_{k=1}^K t_{kn} \ln y_k(x_n, w).$$

This expression is precisely the cross-entropy error function:

$$E(w) = -\sum_{n=1}^N \sum_{k=1}^K t_{kn} \ln y_k(x_n, w).$$

Therefore, maximizing the likelihood of the multi-class neural network is exactly equivalent to minimizing the cross-entropy loss.

2. Binary Classification with Noisy Labels

Consider a binary classification problem with targets $t \in \{0, 1\}$ and network output

$$y(x, w) = p(t = 1 \mid x).$$

Assume that with probability ϵ the class label is flipped (i.e., the observed label is incorrect). Then the probability of observing a target value t given input x becomes:

$$p(t = 1 \mid x) = (1 - \epsilon) y(x, w) + \epsilon (1 - y(x, w)),$$

$$p(t = 0 \mid x) = (1 - \epsilon) (1 - y(x, w)) + \epsilon y(x, w).$$

These two expressions can be written compactly as:

$$p(t \mid x) = \left[(1 - \epsilon) y + \epsilon (1 - y) \right]^t \left[(1 - \epsilon) (1 - y) + \epsilon y \right]^{1-t}.$$

Assuming i.i.d. data, the likelihood over the dataset is

$$\mathcal{L}(w) = \prod_{n=1}^N p(t_n \mid x_n).$$

Taking the negative log-likelihood yields the error function:

$$E(w) = -\sum_{n=1}^N \left[t_n \ln((1 - \epsilon) y_n + \epsilon (1 - y_n)) + (1 - t_n) \ln((1 - \epsilon) (1 - y_n) + \epsilon y_n) \right],$$

where $y_n = y(x_n, w)$.

Verification for $\epsilon = 0$

If $\epsilon = 0$, the expressions reduce to

$$p(t = 1 \mid x) = y, \quad p(t = 0 \mid x) = 1 - y,$$

and the error function becomes

$$E(w) = - \sum_{n=1}^N \left[t_n \ln y_n + (1 - t_n) \ln(1 - y_n) \right],$$

which is exactly the standard binary cross-entropy loss.

Interpretation

For $\epsilon > 0$, the model does not assign zero probability to apparently incorrect labels. Instead, the predicted probability is smoothed toward 0.5. This makes the loss function more robust to label noise compared to the standard cross-entropy formulation.

3. Targets in $\{-1, 1\}$ and Output in $[-1, 1]$

Now consider a binary classification problem with target values

$$t \in \{-1, 1\},$$

and a network output $y(x, w)$ satisfying

$$-1 \leq y(x, w) \leq 1.$$

To interpret the output probabilistically, we define

$$p(t = 1 \mid x) = \frac{1 + y(x, w)}{2}, \quad p(t = -1 \mid x) = \frac{1 - y(x, w)}{2}.$$

For a single data point, the likelihood is therefore

$$p(t \mid x) = \left(\frac{1 + y}{2} \right)^{\frac{1+t}{2}} \left(\frac{1 - y}{2} \right)^{\frac{1-t}{2}},$$

where $t = \pm 1$.

Assuming i.i.d. data, the negative log-likelihood over the dataset becomes

$$E(w) = - \sum_{n=1}^N \left[\frac{1+t_n}{2} \ln \left(\frac{1+y_n}{2} \right) + \frac{1-t_n}{2} \ln \left(\frac{1-y_n}{2} \right) \right],$$

where $y_n = y(x_n, w)$.

This is the cross-entropy loss corresponding to targets in $\{-1, 1\}$.

Equivalent Compact Form

Using algebraic simplification, the error can also be written as

$$E(w) = - \sum_{n=1}^N \ln \left(\frac{1+t_n y_n}{2} \right).$$

Appropriate Output Activation Function

Since the network output must satisfy $-1 \leq y(x, w) \leq 1$, the appropriate activation function is the hyperbolic tangent:

$$y(x, w) = \tanh(a),$$

where a is the pre-activation value.

The tanh function naturally produces outputs in $[-1, 1]$ and is therefore the suitable analogue of the logistic sigmoid for targets in $\{-1, 1\}$.

Question 8 — Stochastic Stability and the “Dead ReLU”

1. Distribution of the Pre-Activation z

Consider a neuron with pre-activation

$$z = w^\top x + b,$$

where $x \in \mathbb{R}^n$ and $w \in \mathbb{R}^n$ are independent random vectors with i.i.d. components:

$$x_i \sim \mathcal{N}(0, \sigma_x^2), \quad w_i \sim \mathcal{N}(0, \sigma_w^2).$$

Mean of z

First compute the expectation:

$$\mathbb{E}[z] = \mathbb{E}[w^\top x] + b = \sum_{i=1}^n \mathbb{E}[w_i x_i] + b.$$

Since w_i and x_i are independent and both have zero mean,

$$\mathbb{E}[w_i x_i] = \mathbb{E}[w_i] \mathbb{E}[x_i] = 0.$$

Therefore,

$$\mathbb{E}[z] = b.$$

Variance of z

We compute the variance:

$$\text{Var}(z) = \text{Var}\left(\sum_{i=1}^n w_i x_i\right).$$

Because the pairs (w_i, x_i) are independent across i ,

$$\text{Var}(z) = \sum_{i=1}^n \text{Var}(w_i x_i).$$

For independent Gaussian variables,

$$\text{Var}(w_i x_i) = \mathbb{E}[w_i^2 x_i^2] = \mathbb{E}[w_i^2] \mathbb{E}[x_i^2] = \sigma_w^2 \sigma_x^2.$$

Hence,

$$\sigma_z^2 = \text{Var}(z) = n \sigma_w^2 \sigma_x^2.$$

Conclusion

Since z is a sum of independent random variables with finite variance, by the Central Limit Theorem it follows a Gaussian distribution:

$$z \sim \mathcal{N}(b, n \sigma_w^2 \sigma_x^2)$$

2. Activation Probability

The neuron output is

$$a = \max(0, z).$$

The neuron is active when

$$a > 0 \iff z > 0.$$

From Part 1, we have

$$z \sim \mathcal{N}(b, \sigma_z^2), \quad \text{where} \quad \sigma_z^2 = n\sigma_w^2\sigma_x^2.$$

Therefore, the activation probability is

$$P(a > 0) = P(z > 0).$$

Standardizing the Gaussian variable,

$$P(z > 0) = P\left(\frac{z - b}{\sigma_z} > \frac{0 - b}{\sigma_z}\right).$$

Since

$$\frac{z - b}{\sigma_z} \sim \mathcal{N}(0, 1),$$

we obtain

$$P(z > 0) = 1 - \Phi\left(\frac{-b}{\sigma_z}\right),$$

where $\Phi(\cdot)$ is the standard normal cumulative distribution function.

Using symmetry of the Gaussian distribution,

$$1 - \Phi(-u) = \Phi(u),$$

we conclude

$$\boxed{P(a > 0) = \Phi\left(\frac{b}{\sigma_z}\right)}$$

Thus, the activation probability depends only on the ratio $\frac{b}{\sigma_z}$.

3. The 3σ Threshold

From Part 2, the activation probability is

$$P(a > 0) = \Phi\left(\frac{b}{\sigma_z}\right).$$

Suppose the bias is initialized as

$$b = -3\sigma_z.$$

Then

$$\frac{b}{\sigma_z} = -3.$$

Therefore,

$$P(a > 0) = \Phi(-3).$$

Using standard Gaussian values,

$$\Phi(-3) \approx 0.00135.$$

Hence,

$$\boxed{P(a > 0) \approx 0.00135}$$

This means that only about 0.135% of inputs activate the neuron.

Expected Number of Active Neurons

If a layer contains $N = 1000$ neurons initialized in this way, the expected number of active neurons is

$$1000 \times 0.00135 = 1.35.$$

Thus, on average,

$$\boxed{\text{approximately 1 to 2 neurons are active.}}$$

This illustrates how strongly negative bias initialization can effectively deactivate almost the entire layer.

4. The Zero-Gradient Proof

Recall that the neuron is defined as

$$z = w^\top x + b, \quad a = \max(0, z),$$

and during backpropagation the derivative of ReLU is

$$\frac{\partial a}{\partial z} = \mathbf{1}_{\{z > 0\}}.$$

Consider a training mini-batch of inputs $\{x^{(1)}, \dots, x^{(m)}\}$. Suppose that for every sample in the mini-batch we have

$$z^{(i)} \leq 0 \quad \text{for all } i = 1, \dots, m.$$

Then for every sample,

$$\frac{\partial a^{(i)}}{\partial z^{(i)}} = 0.$$

Gradient with Respect to the Weights

Using the chain rule, the gradient of the loss L with respect to w is

$$\frac{\partial L}{\partial w} = \sum_{i=1}^m \frac{\partial L}{\partial a^{(i)}} \frac{\partial a^{(i)}}{\partial z^{(i)}} \frac{\partial z^{(i)}}{\partial w}.$$

Since

$$\frac{\partial a^{(i)}}{\partial z^{(i)}} = 0 \quad \text{for all } i,$$

every term in the sum vanishes. Therefore,

$$\frac{\partial L}{\partial w} = 0.$$

Consequently, the weight update

$$\Delta w = -\eta \frac{\partial L}{\partial w}$$

also satisfies

$$\boxed{\Delta w = 0.}$$

Implication

If a neuron has $z \leq 0$ for all samples in a mini-batch, its gradient is exactly zero, and its weights do not change.

This means the neuron cannot adjust its parameters to move into the active region. Unless future mini-batches produce positive pre-activations, the neuron remains permanently inactive.

This phenomenon is known as the *dead ReLU* problem: once a neuron becomes inactive across all samples, gradient descent provides no mechanism to “resurrect” it.

Question 9 — Variance Preservation and Xavier Initialization

1. Symmetric Constraints

Consider a linear layer

$$y^{(l)} = W^{(l)}x^{(l)},$$

where the weights $W_{ij}^{(l)}$ are initialized i.i.d. with

$$\mathbb{E}[W_{ij}] = 0, \quad \text{Var}(W_{ij}) = \sigma^2.$$

Let the input vector $x^{(l)} \in \mathbb{R}^{n_{\text{in}}}$ have zero mean and variance

$$\text{Var}(x_i^{(l)}) = \text{Var}(x).$$

Forward Pass Variance

For a single output unit,

$$y_j^{(l)} = \sum_{i=1}^{n_{\text{in}}} W_{ji} x_i^{(l)}.$$

Since the weights and inputs are independent and zero-mean,

$$\text{Var}(y_j^{(l)}) = \sum_{i=1}^{n_{\text{in}}} \text{Var}(W_{ji} x_i^{(l)}).$$

Because

$$\text{Var}(W_{ji} x_i^{(l)}) = \text{Var}(W_{ji}) \text{Var}(x_i^{(l)}) = \sigma^2 \text{Var}(x),$$

we obtain

$$\text{Var}(y^{(l)}) = n_{\text{in}}\sigma^2\text{Var}(x).$$

To preserve variance across layers, we require

$$\text{Var}(y^{(l)}) = \text{Var}(x),$$

which implies

$$n_{\text{in}}\sigma^2 = 1.$$

Thus,

$$\boxed{\text{Var}(W) = \frac{1}{n_{\text{in}}}}$$

preserves forward-pass variance.

Backward Pass Variance

During backpropagation, let $\delta^{(l)}$ denote the gradient with respect to the pre-activations. The gradient flowing into the previous layer is

$$\delta^{(l-1)} = (W^{(l)})^\top \delta^{(l)}.$$

For a single component,

$$\delta_i^{(l-1)} = \sum_{j=1}^{n_{\text{out}}} W_{ji} \delta_j^{(l)}.$$

By the same reasoning as in the forward case,

$$\text{Var}(\delta^{(l-1)}) = n_{\text{out}}\sigma^2\text{Var}(\delta^{(l)}).$$

To preserve gradient variance, we require

$$n_{\text{out}}\sigma^2 = 1,$$

which gives

$$\boxed{\text{Var}(W) = \frac{1}{n_{\text{out}}}}$$

for backward variance preservation.

2. The Glorot (Xavier) Synthesis

From Part 1, we obtained two constraints:

$$\text{Var}(W) = \frac{1}{n_{\text{in}}} \quad (\text{forward variance preservation}),$$

$$\text{Var}(W) = \frac{1}{n_{\text{out}}} \quad (\text{backward variance preservation}).$$

In general, these two conditions cannot both be satisfied exactly unless $n_{\text{in}} = n_{\text{out}}$. Therefore, we seek a compromise that keeps both forward activations and backward gradients approximately stable.

A natural choice is to set the variance to the average of the two constraints:

$$\text{Var}(W) = \frac{1}{2} \left(\frac{1}{n_{\text{in}}} + \frac{1}{n_{\text{out}}} \right).$$

Combining the fractions gives

$$\text{Var}(W) = \frac{n_{\text{in}} + n_{\text{out}}}{2n_{\text{in}}n_{\text{out}}}.$$

For layers where $n_{\text{in}} \approx n_{\text{out}}$, this is approximately

$$\text{Var}(W) \approx \frac{2}{n_{\text{in}} + n_{\text{out}}}.$$

Thus, the Xavier (Glorot) initialization sets

$$\sigma^2 = \frac{2}{n_{\text{in}} + n_{\text{out}}}$$

which balances forward and backward variance preservation and prevents signal amplification or decay across layers.

3. Depth-Induced Variance

Let the layer gain be defined as

$$G = n \cdot \text{Var}(W),$$

where n denotes the number of inputs to the layer (assuming $n_{\text{in}} = n_{\text{out}} = n$ for simplicity).

From Part 1, the variance of the output of a single layer satisfies

$$\text{Var}(y^{(l)}) = n \text{Var}(W) \text{Var}(x^{(l)}) = G \text{Var}(x^{(l)}).$$

Now consider a network of depth L with linear layers

$$x^{(l+1)} = W^{(l)} x^{(l)}.$$

Applying the variance relation recursively,

$$\text{Var}(x^{(1)}) = G \text{Var}(x^{(0)}),$$

$$\text{Var}(x^{(2)}) = G \text{Var}(x^{(1)}) = G^2 \text{Var}(x^{(0)}),$$

and in general,

$$\text{Var}(x^{(L)}) = G^L \text{Var}(x^{(0)}).$$

Thus, the output variance of the L -th layer is

$$\boxed{\text{Var}(y^{(L)}) = G^L \text{Var}(x^{(0)})}$$

This shows that if $G \neq 1$, the variance either grows or decays exponentially with depth.

4. The Stability Bound

Suppose the initialization contains a small multiplicative error ϵ such that

$$n \cdot \text{Var}(W) = 1 + \epsilon.$$

From Part 3, the layer gain is

$$G = 1 + \epsilon.$$

Therefore, after L layers,

$$\text{Var}(y^{(L)}) = (1 + \epsilon)^L \text{Var}(x^{(0)}).$$

If $\epsilon > 0$, the variance grows exponentially with depth. If $\epsilon < 0$, the variance decays exponentially.

For small ϵ , we can approximate

$$(1 + \epsilon)^L \approx e^{\epsilon L}.$$

Thus,

$$\boxed{\text{Var}(y^{(L)}) = (1 + \epsilon)^L \text{Var}(x^{(0)}) \approx e^{\epsilon L} \text{Var}(x^{(0)})}.$$

This shows that even a small deviation from the ideal condition $n \cdot \text{Var}(W) = 1$ leads to exponential amplification or decay of variance as depth increases.

Hence, deep networks are extremely sensitive to small initialization errors.

5. Numerical Analysis

Suppose the network depth is

$$L = 150,$$

and the initialization error satisfies

$$\epsilon = 0.02.$$

From Part 4, the variance amplification factor after L layers is

$$(1 + \epsilon)^L = (1.02)^{150}.$$

Using the exponential approximation,

$$(1.02)^{150} \approx e^{0.02 \times 150} = e^3 \approx 20.085.$$

More precisely,

$$(1.02)^{150} \approx 19.5.$$

Thus, the output variance is amplified by approximately a factor of

$$\boxed{\text{about } 20.}$$

Implications for FP16 Precision

In 16-bit floating-point (FP16), numerical precision and dynamic range are limited compared to 32-bit floating-point.

An amplification factor of ≈ 20 across 150 layers implies that activations can quickly grow large, increasing the risk of:

- Numerical overflow,
- Loss of precision due to limited mantissa bits,

- Gradient instability.

Because the growth is exponential in depth, even small initialization errors accumulate dramatically in deep networks. This makes variance-preserving initialization (such as Xavier or He initialization) essential for stable training, particularly in low-precision settings like FP16.

Question 10 — Transfer Learning with Pre-trained Networks

In this experiment, we compare a shallow neural network trained from scratch with a deep pre-trained convolutional network on the CIFAR-10 dataset. The goal is to analyze convergence behavior, stability, and generalization performance under different training strategies.

Data Pre-processing

The CIFAR-10 dataset consists of 32×32 RGB images from 10 classes. Images were converted to tensors and normalized using dataset-specific channel statistics:

$$\text{mean} = (0.4914, 0.4822, 0.4465), \quad \text{std} = (0.2470, 0.2435, 0.2616).$$

Training was performed for 10 epochs with mini-batch size 512.

Model Architectures

Three models were implemented:

- **Model 1 (MLP Baseline):** A fully connected network operating on flattened images (dimension 3072).
- **Model 2 (Frozen VGG-11):** Pre-trained VGG-11 on ImageNet with convolutional layers frozen and a new 10-class classifier.
- **Model 3 (Fine-Tuned VGG-11):** Same architecture as Model 2, but all layers were trainable.

All models were trained using the Adam optimizer and cross-entropy loss.

Training Loss

Figure 4 shows the training loss over epochs.

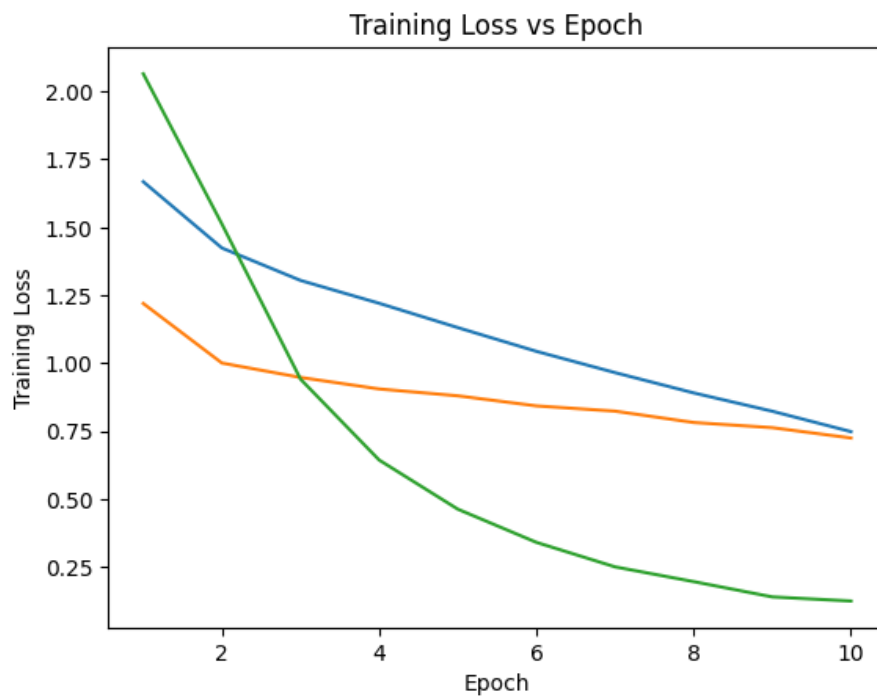


Figure 4: Training loss versus epoch for all three models.

The fine-tuned VGG exhibits the fastest loss reduction, indicating strong optimization capacity. The frozen VGG converges more gradually, while the MLP decreases slowly and remains at higher loss values.

Validation Accuracy

Figure 5 shows validation accuracy over epochs.

Final validation accuracies:

MLP: 54.0%, Frozen VGG: 71.4%, Fine-Tuned VGG: 82.4%.

Discussion

Convergence and Stability

The MLP shows slow and limited improvement due to the absence of spatial inductive bias. Since it processes flattened inputs, it cannot exploit local image structure effectively.

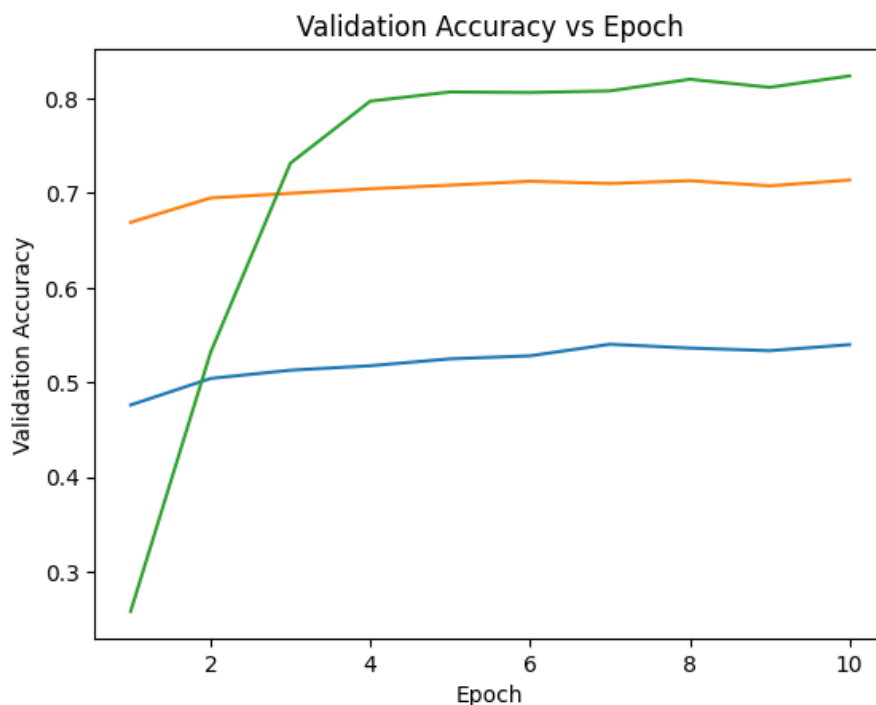


Figure 5: Validation accuracy versus epoch for all three models.

The Frozen VGG begins with strong performance and improves smoothly. Because convolutional layers are pre-trained, feature representations are already meaningful, and only the classifier adapts to CIFAR-10.

The Fine-Tuned VGG initially starts lower but rapidly surpasses the frozen model. Allowing all layers to update enables the network to adapt the pre-trained features to the new dataset, leading to faster descent and better final accuracy.

Pre-trained initialization leads to faster convergence because the model starts in a parameter region already encoding useful visual features, reducing the burden on optimization.

Generalization Gap

The Frozen VGG significantly outperforms the MLP despite having fixed convolutional weights. This is due to the spatial inductive bias of convolutional networks:

- Local receptive fields capture spatial patterns.
- Weight sharing enforces translation invariance.
- Hierarchical feature extraction builds complex representations.

In contrast, the MLP treats pixels independently after flattening, ignoring spatial relationships entirely.

Thus, even without fine-tuning, convolutional features provide a much stronger representation for image data.

Conclusion

This experiment demonstrates that:

- Deep pre-trained networks converge faster and generalize better than shallow networks trained from scratch.
- Fine-tuning further improves performance by adapting features to the target dataset.
- Spatial inductive bias is crucial for image classification tasks.

Transfer learning therefore provides both optimization stability and strong generalization performance, especially when data or training time is limited.